

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration: 1 Hour
Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

*I **JEROLIN MARY R (192411024)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments

- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

ANSWERS:

1. AI-Powered Drones for Emergency Medicine Delivery

1.1 Greedy Best-First Search

Greedy Best-First Search selects the node that appears closest to the destination using a heuristic function ($h(n)$). In this scenario, the straight-line distance to the medicine delivery location can be used as the heuristic.

The drone will therefore prefer a path that appears geographically closest to the destination, without considering the cost already spent reaching that location.

Strengths:

- Makes decisions quickly.
- Requires less computation than A*.
- Useful when fast route selection is important.
- Can work reasonably well when the heuristic is accurate.

Weaknesses:

- May select a route with high weather or terrain costs.
- Does not consider the cost already incurred.
- Can choose a path that becomes blocked.
- Does not guarantee the safest or optimal route.
- An inaccurate heuristic can lead the drone in the wrong direction.

Therefore, Greedy Best-First Search is fast but may make poor decisions in an uncertain environment.

1.2 A* Search

A* Search combines the actual cost from the starting point with the estimated cost to the destination.

The evaluation function is:

$$\begin{aligned} &[\\ f(n) &= g(n) + h(n) \\ &] \end{aligned}$$

Where:

- ($g(n)$) = actual cost of reaching node (n)
- ($h(n)$) = estimated cost from (n) to the destination
- ($f(n)$) = total estimated cost

For the drone, ($g(n)$) can include distance, terrain difficulty, weather conditions, and risk. The heuristic ($h(n)$) can represent the remaining straight-line distance.

Thus, A* does not simply select the closest location. It selects a route that provides a good balance between the cost already incurred and the estimated remaining cost.

1.3 Impact of Heuristic Accuracy

The accuracy of the heuristic has a major effect on both algorithms.

In **Greedy Best-First Search**, the heuristic is the main factor used for selecting nodes. Therefore, an inaccurate heuristic can cause the drone to choose a poor route.

In A*, the heuristic is combined with the actual path cost. Therefore, A* is generally less dependent on the heuristic alone.

If the heuristic is admissible, meaning it does not overestimate the actual remaining cost, A* can guarantee an optimal solution under the standard assumptions.

1.4 Effect of Dynamic Changes

Blocked roads and changing weather can make a previously selected route unusable. When this happens, the search algorithm must reconsider the route and find an alternative.

Dynamic changes can:

- Increase the number of nodes that must be explored.
- Cause previously calculated routes to become invalid.
- Require the algorithm to replan.
- Increase computation time.

Greedy Best-First Search may react quickly but can repeatedly choose routes that appear close but are blocked or risky. A* can use updated movement costs and search for a better alternative, although repeated replanning increases computational requirements.

1.5 Recommended Algorithm

*A Search with dynamic replanning** is the most suitable choice.

Greedy Best-First Search is faster in some cases, but it ignores the actual cost of the route. Since the drone must consider terrain, weather, delay, and safety, A* provides a better balance.

A* is preferable because it considers:

- Actual movement cost.
- Estimated distance.
- Route quality.
- Safety and risk.
- Optimality.

For a real emergency system, a dynamic version of A* would be preferred so that the drone can recalculate its route whenever new information about blocked paths or weather becomes available.

2. Smart City Traffic Signal Optimization

2.1 Formulation as an Optimization Problem

The traffic signal problem can be formulated as an optimization problem where the AI searches for the best signal timings across hundreds of intersections.

The objective is to minimize:

$$\begin{aligned} &[\\ \text{Cost} &= \text{Waiting\ Time} + \text{Fuel\ Consumption} + \text{Traffic\ Congestion} \\ &] \end{aligned}$$

The **decision variables** are the signal timings at each intersection.

The **constraints** can include:

- Minimum and maximum green-light duration.
- Pedestrian crossing time.
- Safety requirements.
- Coordination between nearby intersections.
- Traffic flow requirements.

Traditional search methods become inefficient because the number of possible signal configurations becomes extremely large as the number of intersections increases. Additionally, traffic conditions continuously change.

2.2 Hill Climbing

Hill Climbing begins with an initial traffic signal configuration. It evaluates neighboring configurations and moves to a configuration that improves the objective function.

For example:

Initial timing → Change signal timing → Measure traffic → Keep better solution

This process continues until no neighboring solution provides an improvement.

Advantages:

- Simple to implement.
- Requires relatively little memory.
- Can find improvements quickly.
- Suitable for optimization problems.

Limitations:

- Can become trapped in local optima.
- Can stop at plateaus.
- Can have difficulty with ridges.

- The result can depend on the initial solution.

2.3 Local Maxima, Plateaus and Ridges

Local Maximum/Minimum

A local optimum is a solution that is better than its nearby solutions but is not the best solution overall.

Example: A traffic signal configuration may reduce congestion at one intersection but increase congestion across the entire road network.

Plateau

A plateau occurs when many neighboring solutions have almost the same objective value.

Example: Changing a traffic light from 30 seconds to 31 seconds may produce almost no difference in traffic flow. Hill Climbing may not know which direction to take.

Ridge

A ridge occurs when the best solution requires several coordinated changes rather than one individual change.

Example: Improving traffic flow may require changing the timings of several connected intersections together.

2.4 Simulated Annealing and Genetic Algorithms

Simulated Annealing can sometimes accept a worse solution temporarily. This allows the algorithm to escape local optima and explore other areas of the search space.

As the algorithm progresses, the probability of accepting worse solutions decreases.

Genetic Algorithms maintain a population of possible traffic signal configurations. The solutions are evaluated using a fitness function, and better solutions are selected for reproduction.

Genetic Algorithms use:

- Selection
- Crossover
- Mutation

This allows the algorithm to explore multiple possible solutions instead of following only one path.

2.5 Recommended Approach

A **Genetic Algorithm** is a suitable choice for this scenario because the problem involves hundreds of intersections and a very large search space.

It can:

- Explore multiple solutions simultaneously.
- Avoid becoming permanently trapped in a local optimum.
- Adapt to changing traffic patterns.
- Handle many decision variables.

Therefore, a **Genetic Algorithm combined with real-time traffic data** would provide good scalability and adaptability.

3. Autonomous Mars Rover

3.1 Need for an Online Search Agent

An offline search algorithm normally assumes that the environment and problem information are available before searching.

The Mars rover does not have a complete map. It discovers terrain and obstacles while moving.

Therefore, it requires an **online search agent**.

The rover can:

1. Observe its surroundings.
2. Make a decision.
3. Move.
4. Discover new information.
5. Update its knowledge.
6. Recalculate the next action.

This makes online search more appropriate than relying on a completely precomputed route.

3.2 Partial Observability and Dynamic Environments

The rover has limited sensing capabilities and cannot observe the entire environment.

Major challenges include:

- Unknown rocks and craters.
- Limited sensor range.
- Uncertain terrain conditions.
- Limited battery power.
- Communication delays.
- Incomplete maps.
- Unexpected hazards.

Because the rover does not have complete information, it must make decisions using its current knowledge and update those decisions when new information becomes available.

3.3 Building and Updating the Internal Model

The rover continuously collects information through sensors.

The process can be represented as:

Sense → Analyze → Update Map → Plan → Move → Sense Again

For example, if the rover detects a crater that was not previously known, it updates its internal map and marks that area as unsafe.

It then searches for an alternative route to the target location.

3.4 Comparison with Other Search Strategies

Search Type	Knowledge	Characteristic
Uninformed Search	No heuristic	Explores without additional guidance
Informed Search	Uses heuristic information	Guides search toward the goal
Online Search	Environment is initially unknown	Learns while interacting with the environment

Uninformed and informed searches can be useful when the environment is already known. However, the rover needs to discover its environment while operating, making online search more appropriate.

3.5 Recommended Approach

An **online informed search agent** is the most suitable approach.

The rover can combine:

- Real-time sensor information.
- Heuristic estimates.
- Continuous replanning.
- Hazard detection.
- Safety constraints.

This approach allows the rover to adapt to unknown terrain while reducing unnecessary movement and avoiding dangerous areas.

4. University Exam Timetable Using CSP

4.1 Formulation as a Constraint Satisfaction Problem

A Constraint Satisfaction Problem (CSP) consists of:

1. Variables
2. Domains
3. Constraints

For the examination timetable, each course can be represented as a variable.

For example:

```
[  
X_1 = Mathematics  
]
```

```
[  
X_2 = Data\ Structures  
]
```

```
[  
X_3 = Operating\ Systems  
]
```

Variables

The variables represent the examinations that need to be scheduled.

Domains

The domain contains the possible combinations of examination time slots and halls.

For example:

```
[  
D(X_1)={Monday\ 9AM,\ Monday\ 2PM,\ Tuesday\ 9AM}  
]
```

Constraints

The timetable must satisfy:

- Students cannot have overlapping examinations.
- A hall cannot exceed its capacity.
- Certain subjects must be scheduled before others.
- Faculty must be available.
- Special accommodation requirements must be satisfied.

A valid timetable is produced only when all constraints are satisfied.

4.2 Backtracking Search

Backtracking assigns values to variables one at a time.

The process is:

Select exam → Assign slot → Check constraints → Continue

If a conflict occurs, the algorithm goes back to a previous assignment and tries another value.

For example, suppose Mathematics and Physics have common students and are assigned to the same time. This violates a constraint.

The algorithm rejects that assignment and tries another available time slot.

Backtracking allows the system to systematically explore possible timetables without checking every complete timetable.

4.3 Constraint Propagation and Forward Checking

Forward checking improves backtracking by detecting conflicts earlier.

After assigning a time slot to one examination, incompatible values are removed from the domains of related examinations.

For example, if Mathematics is scheduled on Monday at 9 AM, that slot can be removed from the domain of other examinations taken by the same students.

This:

- Reduces the search space.
- Detects conflicts early.
- Reduces unnecessary backtracking.
- Improves computational efficiency.

4.4 Challenges and Best Strategy

Large universities may have hundreds of courses, thousands of students, many halls, faculty restrictions, and special accommodations.

As the number of variables and constraints increases, simple backtracking becomes computationally expensive.

A suitable strategy is:

Backtracking + Forward Checking + Constraint Propagation + Heuristics

A useful heuristic is **Minimum Remaining Values (MRV)**, which selects the variable with the smallest remaining domain first.

This helps identify difficult scheduling decisions early and improves scalability.

5. Strategic Game AI Using Minimax and Alpha-Beta Pruning

5.1 Minimax Model

Minimax models the game as a tree of possible moves.

There are two types of players:

- **MAX:** The AI tries to maximize its score.
- **MIN:** The human opponent tries to minimize the AI's score.

The algorithm explores possible moves and assumes that the opponent will make the best possible response.

The AI chooses the move that provides the best outcome against the opponent's strongest possible response.

5.2 Computational Challenges

A game can have a very large number of possible moves.

If:

- (b) = branching factor
- (d) = search depth

then the time complexity of Minimax is:

[
 $O(b^d)$
]

This means that increasing the depth or number of possible moves causes the number of states to increase exponentially.

For a real-time game, searching the entire tree is usually impossible because the AI must make decisions within strict time limits.

5.3 Alpha-Beta Pruning

Alpha-Beta pruning improves Minimax by removing branches that cannot affect the final decision.

It maintains two values:

- **Alpha:** The best value currently guaranteed for MAX.
- **Beta:** The best value currently guaranteed for MIN.

If the algorithm discovers that a branch cannot produce a better result than an already explored branch, it stops evaluating that branch.

For example, if MAX already has a move guaranteeing a score of 8 and another branch can only produce a maximum score of 6, there is no need to explore that branch further.

Alpha-Beta pruning therefore produces the **same decision as Minimax while evaluating fewer nodes**.

Its complexity is:

Worst case:

[
 $O(b^d)$
]

Best case:

[
 $O(b^{\{d/2\}})$
]

This allows the AI to search deeper within the same time limit.

5.4 Evaluation Function

The complete game tree cannot usually be explored, so an evaluation function estimates the quality of a game state.

For example:

[
Evaluation =
5(Unit\ Strength)
+4(Resource\ Advantage)
+3(Map\ Control)
+3(Building\ Advantage)
-4(Damage\ Taken)
]

The evaluation function can consider:

- Number and strength of units.
- Available resources.
- Territory controlled.
- Buildings and technology.
- Unit health.
- Damage taken.
- Strategic position.
- Immediate threats.

The weights determine the importance of each factor.

A higher evaluation value represents a more favorable state for the AI.

5.5 Recommended Strategy

For a real-time strategy game, the best approach is:

Minimax + Alpha-Beta Pruning + Depth-Limited Search + Evaluation Function

The AI should search only to a practical depth and use the evaluation function when the depth limit is reached.

Iterative deepening can also be used. The AI first searches to a small depth and then progressively increases the depth until the available time is almost exhausted.

This provides a good balance between:

- Optimality
- Search depth
- Computational efficiency
- Real-time response

Overall Recommended Techniques

Scenario	Recommended Technique
Emergency Drone	<i>A with Dynamic Replanning*</i>
Traffic Optimization	Genetic Algorithm
Mars Rover	Online Informed Search
Exam Timetabling	CSP + Backtracking + Forward Checking
Game AI	Minimax + Alpha-Beta Pruning + Depth Limiting